# Research Plan: Architecting a Host-Integrated I/O Uncore

This document revises and elaborates the original research plan based on our detailed discussions. It clarifies **scope, ordering, assumptions, and evaluation methodology**, and explicitly separates **architectural motivation** from **silicon feasibility**.

---

## 0. Research Thesis

**Thesis:** At ultra-high IOPS, host-side overhead—especially NVMe queue execution, CQ polling, MMIO doorbells, and DRAM metadata traffic—dominates energy and scalability. Moving the *per-I/O execution plane* into a CPU-integrated I/O uncore (using private SRAM and simple hardware state machines) decouples IOPS scaling from CPU cores and delivers order-of-magnitude IOPS/W improvements.

This work proceeds in **phases**, starting with *measurement and modeling* on commodity systems, and only later addressing *RTL and silicon feasibility*.

---

## 1. Motivation

Storage-next SSDs are trending toward multi–ten-million random IOPS per device, yet hosts routinely cap near ~0.5–1.5M small-block IOPS per core once doorbell MMIO, CQ polling of device-written cachelines, DRAM metadata traffic, fences, and translation overheads are counted. Peak demonstrations (5–10M IOPS/core) rely on full-core busy polling, perfect NUMA locality, and aggressive batching—conditions that are fragile, power-inefficient, and non-scalable across tenants.

This mirrors the pre–integrated-memory-controller era: CPUs once accessed memory and I/O through off-die chipsets, creating a latency and power wall. The solution was integration. Today's host-storage path faces an analogous wall, and the natural next step is to integrate **I/O execution** into the CPU uncore.

SPDK and io_uring remove syscalls and interrupts, but architectural ceilings remain: per-I/O MMIO, CQ polling, ordering fences, and DRAM metadata churn still cost ~700–1200 cycles per I/O in steady state. At tens of MIOPS, host energy per I/O—not device latency—becomes the limiting factor.

**Industry context and urgency.** As storage-next SSD controllers push device-level parallelism and IOPS higher, overall system performance increasingly depends on how effectively host processors can *consume* those IOPS. Without corresponding advances on the host side, additional SSD capability yields diminishing returns: IOPS are stranded behind CPU cores, DRAM bandwidth, and MMIO limits. This creates a widening device–host utilization gap and shifts the bottleneck from media to the host execution path. The proposed I/O uncore directly targets this gap, aligning host capabilities with next-generation SSD throughput so device innovations translate into end-to-end gains.

**Beyond storage-next: inevitability from mainstream SSD scaling.** This pressure is not limited to specialized "storage-next" devices. Mainstream NVMe SSD design has historically aimed to saturate available PCIe bandwidth using controller parallelism and flash concurrency. As PCIe generations advance, even conventional x4 NVMe SSDs are expected to deliver aggregate bandwidths on the order of tens of GB/s, corresponding to **low-teens million random 4 KB IOPS per device**. With only a handful of such drives, aggregate node-level IOPS can exceed **50 M+ IOPS** without assuming smaller block sizes or exotic devices. In this regime, software-based NVMe queue execution—regardless of kernel bypass, polling, or batching—fails to scale economically, as per-I/O host execution cost grows linearly with IOPS. Consequently, host-side hardware execution of NVMe queue semantics becomes a **minimal architectural requirement** rather than an optional optimization, making an I/O uncore increasingly inevitable even for systems built from otherwise conventional SSDs.

**Implications for DPUs and GPU-centric systems.** The same host-side execution bottlenecks have motivated GPU- and DPU-centric approaches (e.g., GPU-direct storage pipelines and smart NICs) that offload polling and orchestration away from CPU cores. However, these approaches primarily *bypass* the host rather than fixing the root cause: inefficient queue execution and signaling. Integrating an I/O uncore into a DPU-class SoC (e.g., BlueField) would fundamentally change this trade-off. By providing SRAM-resident SQ/CQ execution, doorbell aggregation, and CQ suppression on the DPU, such a platform could efficiently service highly concurrent, fine-grained random reads demanded by AI and embedding-heavy workloads—without requiring GPUs to perform pointer-heavy control tasks. This would significantly expand the DPU's addressable market from networking and coarse-grained storage offload into AI data pipelines, while preserving energy efficiency and generality across CPU- and GPU-attached consumers.

---

# 2. Target Workload Regime (Explicit Scope)

This work primarily targets **embedding-heavy pipelines** (e.g., vector databases, retrieval for AI workloads) with the following properties:

- ANN index and metadata are **memory-resident**
- SSD traffic is dominated by **random embedding payload fetches**
- DMA targets are **reused, pinned huge-page buffer pools** (CPU or GPU staging)

In this regime:

- IOMMU miss rates are low
- Translation overhead is secondary
- **Queue execution, polling, MMIO, and DRAM metadata traffic dominate**

Accordingly, the I/O uncore design prioritizes queue execution and signaling efficiency; IOMMU assist remains a **secondary optimization**.

---

# 3. Ordered Responsibilities of the I/O Uncore

## 1️⃣ SRAM-Resident SQ/CQ Execution (Foundation)

**Principle:** Execute NVMe submission and completion queue *control logic* entirely from uncore-private SRAM, keeping head/tail pointers and per-queue metadata off host DRAM.

**Rationale:** Queue control state is touched every I/O; removing it from DRAM eliminates coherence traffic, polling, and per-I/O metadata bandwidth, which dominate host cost at scale.

---

## 2️⃣ Doorbell Aggregation & CQ Suppression

**Principle:** Replace per-I/O MMIO doorbells and fine-grain DRAM-visible completion exposure with batched, rate-controlled signaling.

**Rationale (clarified).** CQ suppression does **not** rely on the existence of empty polling iterations. Instead, it increases *completion density* and removes DRAM-visible queue state from the execution path. Even when SPDK drains multiple completions per poll at very high IOPS, the dominant host cost comes from cacheline invalidations, pointer coherence, and MMIO ordering associated with per-I/O CQ updates. By accumulating completions in SRAM and exposing them to DRAM in batches, the uncore dramatically reduces CQ cacheline churn and coherence traffic per completed I/O. This lowers cycles per I/O and enables higher IOPS per CPU core, independent of whether queues are frequently empty.

In the transparent mode (no SPDK changes), SPDK continues to poll DRAM CQs, but each productive poll drains a larger batch of completions and DRAM-visible CQ state changes far less often. In the poll-lite mode (minimal SPDK NVMe library enablement), SPDK avoids scanning DRAM CQs altogether by polling a lightweight uncore readiness signal or using interrupt-per-batch delivery, eliminating empty scanning overhead in multi-queue configurations.

---

## 3️⃣ DMA Orchestration & Pipelining

**Principle:** Convert SQEs into deeply pipelined DMA operations in hardware, with explicit backpressure and cache-bypass control.

**Rationale:** After queue overhead is removed, efficient DMA issue and overlap dominate throughput and energy for small random reads.

---

## 4️⃣ Hardware QoS & Fairness at Line Rate

**Principle:** Enforce per-queue and per-tenant scheduling and quotas directly in the uncore, independent of CPU intervention.

**Rationale:** At ultra-high IOPS, software schedulers are too coarse; fairness must be enforced at the same granularity as I/O issuance.

---

### 5️⃣ Telemetry & Observability

**Principle:** Collect per-flow counters and latency histograms in hardware and expose them as first-class signals to software.

**Rationale:** Hardware-level visibility is required to validate fairness, tune batching policies, and reason about IOPS/W at scale.

---

### 6️⃣ IOMMU Assist (Secondary Optimization)

**Principle:** Opportunistically assist address translation via lookahead and page-walk caching, without replacing IOMMU authority.

**Rationale:** For embedding-heavy pipelines using reused, pinned huge-page buffers, translation misses are rare; predictability and queue efficiency dominate.

---

## 4. NVMe Compliance and SRAM Sizing

This section explains **how the I/O uncore remains fully NVMe-compliant while moving execution into SRAM**, and **why the proposed SRAM sizes are sufficient and realistic**. This is a critical section for avoiding reviewer concerns about correctness, compatibility, and feasibility.

---

### 4.1 NVMe Compliance: Architectural Truth vs Execution Truth

A key design principle is the separation between:

- **Architectural truth**: what software and the NVMe specification observe
- **Execution truth**: what actually drives per-I/O progress in hardware

In this design:

- NVMe Submission Queues (SQs) and Completion Queues (CQs) **continue to exist in host DRAM**, exactly as defined by the NVMe specification.
- The operating system, drivers, and tools observe standard NVMe behavior: queue memory in DRAM, head/tail semantics, MSI-X interrupts, and error handling are unchanged.

The I/O uncore does **not** redefine NVMe semantics. Instead, it **caches and proxies queue execution**:

- Per-I/O queue *control state* (head/tail, credits, moderation counters) lives in uncore-private SRAM.
- DRAM-resident rings are treated as **compatibility mirrors**, updated at bounded intervals rather than per I/O.

This preserves:

- software compatibility (POSIX, io_uring, SPDK unchanged),
- debuggability (queue state remains inspectable in memory),
- virtualization correctness (guest-visible NVMe model intact).

---

## 4.1.1 Placement in the Linux I/O Stack

The I/O uncore is positioned **below the Linux block layer and behind the NVMe driver**, so applications and higher kernel layers never interact with it directly. Linux continues to act as the **control plane**, while the uncore implements the **execution plane** for NVMe queue advancement and signaling.

From software's perspective, the system still exposes a standard NVMe controller. The only software component aware of the uncore is the NVMe driver, which binds queues and configures offload policies during initialization.

Conceptually, the stack is organized as follows:

```
Applications
  ↓
Filesystems
  ↓
blk-mq / io_uring
  ↓
NVMe driver        ← abstraction boundary
  ↓
I/O uncore         ← execution plane (hidden)
  ↓
NVMe device
```

Above the NVMe driver, the Linux kernel and applications are unchanged. Below the driver, the uncore absorbs hot-path work that would otherwise be performed repeatedly in software, such as queue pointer updates, doorbell signaling, and completion polling.

This placement mirrors existing Linux abstractions for DMA engines, interrupt controllers, and IOMMUs: hardware accelerators are **consumed through drivers**, not exposed as new programming models.

---

## 4.1.2 Placement in the SPDK I/O Stack

SPDK bypasses the Linux block layer and implements NVMe driver logic entirely in user space. In this case, the I/O uncore is positioned **below the SPDK NVMe library and above the PCIe root complex**, remaining invisible to SPDK applications.

SPDK applications issue I/O through the SPDK NVMe library, which formats SQEs, writes them into host memory, rings doorbells, and polls for completions. With an I/O uncore present, these same actions occur unchanged; however, the *hardware consequences* of those actions are altered by the uncore.

Conceptually, the SPDK stack is organized as follows:

```
SPDK application
  ↓
SPDK NVMe library   ← abstraction boundary
  ↓
I/O uncore          ← execution plane (hidden)
  ↓
PCIe root complex
  ↓
NVMe device
```

Only the SPDK NVMe library is aware of the uncore, and only at queue setup time. During initialization, the library registers SQ/CQ base addresses, queue depth, and doorbell locations with the uncore and enables offload policies. After this one-time configuration, no per-I/O software interaction with the uncore is required.

Crucially, SPDK applications are completely unchanged. They continue to submit and complete I/O using standard SPDK APIs, while the uncore executes NVMe queue semantics in hardware by observing SQ writes, MMIO doorbells, and CQ DMA events on the host I/O fabric. This preserves SPDK's programming model while removing CPU, DRAM, and MMIO overhead from the hot path.

---

## 4.2 What Lives in SRAM vs DRAM

**Always resident in uncore SRAM (hot, per-I/O state):**

- SQ head pointer
- CQ head and tail pointers
- Outstanding command counters / credits
- CQ moderation counters and timers
- Small FIFOs for decoded SQEs and pending CQEs

These structures are touched **every I/O** and dominate coherence and DRAM traffic if left in memory.

**Streamed through SRAM (working-set buffers):**

- A small window of prefetched SQEs
- A small buffer of completed CQEs awaiting batch writeback

These are single-use tokens and do not require full-queue residency.

**Remain in host DRAM (architectural state):**

- Full SQ and CQ ring buffers
- DRAM-visible head/tail values (updated in batches)
- Error and recovery state required by NVMe

This division ensures that SRAM is used only for **high-reuse control state**, keeping area and power modest.

---

## 4.3 NVMe Ring Handling Modes

The three modes describe **how DRAM-visible NVMe rings are maintained for architectural correctness** while execution moves into the uncore. They are **not three different architectures**, but **three compatibility/performance points** on the same architecture.

**Research focus clarification.** This work does **not** study all three modes equally:

- **Mode A (Shadowed Rings)** is the **primary focus** of the research and evaluation, as it demonstrates substantial benefits with *unchanged applications and frameworks*.
- **Mode B (Mailbox / Assisted Submission)** is **optional** and used sparingly to illustrate incremental gains with minimal enablement.
- **Mode C (Logical Rings in SRAM)** is treated as an **architectural upper bound** and a guide for silicon sizing, not as a primary experimental target.

**Key clarification.** Regardless of mode, the I/O uncore executes queue semantics in SRAM. The modes differ only in **how aggressively DRAM rings are updated and exposed to software**.

---

---

**Mode A: Shadowed Rings (Default; Transparent to Software)**

- Software (kernel NVMe or SPDK) writes SQEs to DRAM and polls DRAM CQs as usual.
- The uncore observes SQ writes, doorbells, and CQ DMA writes, and executes queue semantics in SRAM.
- DRAM-visible SQ/CQ pointers and CQEs are updated **in batches** for compatibility.

**What this mode demonstrates.**

- Zero application changes.
- No mandatory SPDK library changes.
- Lower-bound benefit from reduced DRAM-visible pointer execution, CQ cacheline churn, and MMIO ordering—even when SPDK continues to poll DRAM.

This mode anchors the **drop-in deployability** story.

---

**Mode B: Mailbox / Assisted Submission (Optional Enablement)**

- Software submits SQEs via a lightweight mailbox or shadow interface to the uncore.
- The uncore synthesizes NVMe-compliant SQEs into DRAM rings as needed.

**What this mode demonstrates.**

- Further reduction in DRAM writes on the submission path.
- Cleaner batching and backpressure control.
- A stepping stone for environments willing to accept minimal driver/library enablement.

This mode is **optional** and primarily a performance/efficiency knob, not a requirement.

---

**Mode C: Logical Rings in SRAM (Maximum Suppression)**

- The uncore maintains logical SQ/CQ rings entirely in SRAM.

- DRAM rings are updated only at bounded synchronization points (e.g., on interrupts, quiescence, or debugging).

**What this mode demonstrates.**

- Upper-bound efficiency when software cooperates (e.g., poll-lite SPDK or interrupt-per-batch).
- Elimination of DRAM-driven execution and scanning.

This mode illustrates the **ceiling** of the architecture and is intended for tightly controlled environments.

---

**Why keep all three modes in the paper.**

- Mode A proves the architecture is valuable **even with unchanged software**.
- Mode B shows incremental gains with minimal enablement.
- Mode C establishes the **maximum achievable efficiency** and guides silicon sizing.

Together, the modes present a **continuum from transparency to peak efficiency**, avoiding an all-or-nothing claim while remaining fully NVMe-compliant.

---

## 4.4 SRAM Sizing: Why the Numbers Are Reasonable

The SRAM sizing model is driven by **active queue working sets**, not total queue depth.

**Rule of thumb:**

- ~4 KB per active SQ (head/tail, decoded SQEs, credits)
- ~4 KB per active CQ (head/tail, CQE buffer, moderation state)
- ~2 MB per tile for shared structures (DMA FIFOs, schedulers, telemetry, translation caches)

This yields:

- ~8 KB per active queue pair (QP)
- ~3–11 MB SRAM per tile for 10–80M IOPS
- ~16–32 MB SRAM only for extreme headroom scenarios

Crucially:

- SRAM scales with **concurrent active QPs**, not device queue depth
- Embedding-heavy workloads typically use **few hot queues**, even at high IOPS

---

## 4.5 Why SRAM (Not DRAM, Not Caches)

Uncore-private SRAM is chosen because it:

- avoids cache coherence traffic entirely,
- provides deterministic, low-latency access,
- consumes orders-of-magnitude less energy per access than DRAM,
- scales naturally within modern I/O chiplets.

Using shared LLC instead would reintroduce coherence and eviction effects; using uncore-local DRAM would reintroduce row-buffer and energy costs.

---

## 4.6 Failure Handling and Correctness

If the uncore stalls, resets, or is disabled:

- Software-visible NVMe rings in DRAM remain correct.
- The system can fall back to standard software-driven NVMe execution.

This fail-safe property is essential for deployability and debugging.

---

## 4.7 SRAM Sufficiency Conditions and Scaling Caveats

The proposed SRAM sizing (single-digit to low–tens of MB per tile) is **sufficient and realistic under explicit conditions**, but it is not unconditional.

**Sufficiency conditions:**

- SRAM is provisioned per **concurrently active** queue pair (QP), not per allocated or configured QP.
- Each active QP requires only control state and small staging windows (head/tail, credits, limited SQE/CQE buffers), not full ring mirroring.
- Doorbell aggregation and CQ moderation are enabled, so queue visibility and completion exposure are batched rather than per-I/O.
- Embedding-heavy workloads concentrate load on a relatively small number of hot queues per device or tenant.

Under these conditions, 128–1024 active QPs map naturally to ~3–11 MB of SRAM per tile, with additional headroom achieved by adding tiles rather than scaling a single tile monolithically.

**Scaling caveats:**

- If thousands of QPs are simultaneously hot at line rate, SRAM capacity grows linearly with active QPs and may exceed low–tens of MB per tile.
- Aggressive low-latency policies (very small batching thresholds) increase SRAM bandwidth and port pressure even if capacity remains sufficient.
- In such regimes, scalability is addressed via tighter multiplexing of QP state, hierarchical scheduling, or replication across multiple I/O tiles.

These caveats do not undermine feasibility; they define the **operating envelope** in which SRAM-resident SQ/CQ execution delivers maximum benefit.

---

# 5. Evaluation Plan

This section is reviewed and strengthened to ensure the evaluation is **sufficient to support the inevitability thesis**, addresses likely reviewer objections, and cleanly separates *measurement*, *validation*, and *feasibility*. Additional experiments and checks are added where necessary.

# Phase 1: Measurement & Modeling on Commodity Hardware

**Why SPDK first.** Phase 1 intentionally starts with **SPDK as the primary evaluation stack**, not because it is easier to modify, but because it represents the *best-case software baseline* for NVMe queue execution. SPDK already eliminates syscalls, interrupts, and kernel scheduling overhead, relying on user-space polling and direct device access. Any remaining overhead observed in SPDK therefore reflects **fundamental host execution costs** (polling, queue metadata traffic, MMIO, coherence), not Linux or driver inefficiencies. Demonstrating large per-I/O overheads even with SPDK provides the strongest possible motivation for moving queue execution into hardware.

Accordingly, Phase 1 treats SPDK as the reference point. Extensions to io_uring and the kernel NVMe stack are secondary and can be evaluated later once the core argument is established.

**Purpose.** Phase 1 establishes the *architectural motivation* for the I/O uncore by quantitatively identifying where host-side cost is spent today. The objective is not to maximize IOPS, but to **measure per-I/O overheads** that fundamentally limit scaling.

**Goal.** Quantify how much CPU work, DRAM traffic, and MMIO activity are attributable specifically to NVMe SQ/CQ execution and polling, as opposed to payload movement.

---

## 1.1 Experimental setup

**Hardware.**

- Laptop or desktop system (commodity is fine)
- Commodity NVMe SSD (consumer or enterprise)
- Prefer AC power and performance mode (disable aggressive power saving for repeatability)

**Software stack (primary).**

- **SPDK as the reference baseline** (best-case software)
- Start with SPDK's own benchmarks: `spdk_nvme_perf` or `bdevperf` (fio plugin optional later)

**Workload shape.**

- Random reads: 4 KB and 16 KB (represent embedding fetch blocks)
- Queue depth (QD): 32, 64, 128
- Number of qpairs: 1, 4, 16, 64 (or as many as your SSD and CPU can sustain)

**CPU/pinning.**

- Pin the polling thread(s) to dedicated core(s).
- Disable CPU frequency scaling if possible (or record fixed frequency).
- Keep other system activity minimal.

**Steady-state window.**

- Warm up for 10–30s.
- Measure for 30–60s steady state.
- Repeat 3 times and report mean + stddev.

**Deliverable:** a reproducible run script that (1) sets CPU affinity, (2) runs the SPDK workload, and (3) collects counters for the same time window.

## 1.2 Experiment matrix (what to sweep)

To make results interpretable, sweep **one variable at a time**:

1. **QD sweep:** QD ∈ {16, 32, 64, 128} at fixed qpairs
2. **qpair sweep:** qpairs ∈ {1, 4, 16, 64} at fixed QD
3. **IO size sweep:** 4K vs 16K at fixed QD and qpairs
4. **core sweep:** 1 poller core vs 2 vs 4 (if needed to scale)

**Why these sweeps matter:**

- QD controls in-flight concurrency and completion burstiness.
- qpairs control multi-queue scanning pressure and pointer/churn scaling.
- IO size changes payload vs metadata ratio.
- #poller cores captures the "IOPS scales with cores" baseline.

---

## 1.3 Metrics collected

Below is the recommended metric set, grouped as **must-have** vs **nice-to-have**.

### Must-have (minimum publishable)

All normalized per completed I/O during steady state:

1. **IOPS and latency**

- IOPS (overall)
- p50/p99/p99.9 latency (even coarse histogram)

1. **CPU cost**

- total cycles/IO
- instructions/IO
- CPU utilization per poller thread

1. **Polling / scanning intensity**

- calls to completion processing per second (or per IO)
- completions drained per call (average, distribution)
- **poll/scans per completion** = (completion-check calls) / (completions)

1. **DRAM traffic**

- DRAM read bytes/IO
- DRAM write bytes/IO
- **DRAM metadata bytes/IO (approx.)** = max(0, DRAM bytes/IO − payload bytes/IO)

1. **Doorbell activity**

- doorbells/IO (or doorbells/sec)

These are sufficient to justify the architectural claim: “host cycles/IO and DRAM/IO are dominated by queue execution and signaling.”

### Nice-to-have (strengthens attribution)

- LLC-load-misses/IO (or MPKI)
- PCIe traffic counters if available (posted writes as proxy for MMIO)
- Package power (RAPL) → joules/IO (host-side)
- Context-switches/second on poller cores (should be near zero)

---

## 1.4 How to collect the metrics (step-by-step guidance)

This section is intentionally practical. The student should be able to follow it on commodity hardware.

### 1.4.1 Workload runner

Preferred:

- `spdk_nvme_perf` or `bdevperf` with fixed IO size, QD, and qpair count.

Optional (later):

- fio with SPDK plugin for convenience; note that SPDK native apps are usually cleaner for attribution.

**Guidance:** Start with a single SSD, single namespace, single thread, then scale qpairs.

### 1.4.2 CPU cycles, instructions, and CPU utilization

Use `perf stat` around the benchmark process pinned to the poller core(s):

- cycles, instructions
- task-clock (or cpu-clock)

Compute:

- cycles/IO = total cycles / completed IOs
- instr/IO = total instructions / completed IOs

### 1.4.3 Polling/scanning intensity (the missing key metric)

Add one of the following instrumentation methods:

**Method A (preferred): lightweight SPDK counters**

- Count calls to `spdk_nvme_qpair_process_completions()` per second
- Count number of completions returned per call
- Report average and distribution

**Method B: perf sampling attribution**

- Use `perf record` + `perf report` to identify the completion polling function hot spots
- This gives % cycles in polling vs other work

Key derived metrics:

- polls (or completion-check calls) per completion
- completions per poll-call

These metrics directly quantify “how much CPU work is spent just discovering completions.”

### 1.4.4 DRAM bandwidth (reads/writes)

Use a memory bandwidth tool appropriate to the platform:

- Intel PCM (`pcm-memory`) if available
- or uncore IMC PMU counters via `perf` (platform dependent)

Compute:

- DRAM_bytes/IO = (DRAM_read_bytes + DRAM_write_bytes) / IOs
- DRAM_metadata_bytes/IO ≈ max(0, DRAM_bytes/IO − payload_bytes/IO)

**Note:** This is a conservative estimate; it understates metadata overhead when payload is served from cache.

### 1.4.5 Doorbells/IO (MMIO rate)

On commodity systems, direct MMIO counting is hard without instrumentation. Use one of:

**Method A (preferred): software counter at the doorbell write site**

- In SPDK NVMe library, increment a counter where the doorbell MMIO write is performed
- Report doorbells/sec and doorbells/IO

**Method B: proxy metrics**

- If PCIe posted-write counters are available, use as a proxy for doorbells/sec (lower fidelity)

Doorbells/IO is crucial because it captures the part of the overhead that batching can amortize.

### 1.4.6 Latency histograms

Collect at least:

- p50/p99/p99.9 latency

Even coarse bins are acceptable for Phase 1; Phase 2 should tighten this.

### 1.4.7 Optional: host energy per I/O

If RAPL is available:

- measure package energy over the measurement interval
- joules/IO = energy / IOs

This strengthens the IOPS/W argument, even on commodity hardware.

---

## 1.5 Sanity checks (to avoid misleading data)

- Verify poller cores have near-zero context switches.
- Ensure runs are steady-state (IOPS stable over window).
- Check that CPU frequency is stable or recorded.
- Confirm that QD and qpair counts are actually applied.
- Repeat runs and report variance.

**Common failure modes:**

- CPU throttling on laptops (thermal)
- background OS activity on poller core
- mismatched SPDK hugepage configuration

---

## 1.6 Counterfactual modeling

Using the measured data, construct a counterfactual model in which host-side NVMe queue semantics are no longer realized through CPU polling, cache coherence, and DRAM visibility. Instead:

- CQ completion tracking is assumed to occur in SRAM, with DRAM-visible CQ updates batched.
- SQ/CQ head and tail updates are assumed to be SRAM-resident, with DRAM pointers updated only at bounded intervals.
- Doorbell MMIO is assumed to be aggregated rather than issued per I/O.

Importantly, this model applies equally to regimes where queues are frequently non-empty (e.g., 20–40M IOPS devices with many queues). The modeled benefit arises from eliminating DRAM-visible pointer execution, cacheline churn, and MMIO ordering costs per I/O.

Report modeled improvements as:

- cycles/IO reduction
- DRAM metadata bytes/IO reduction
- doorbells/IO reduction
- implied IOPS/core increase (via cycles/sec ÷ cycles/IO)

---

## 1.7 Phase 1 outputs (what to put in the paper)

Phase 1 should produce **3–5 key plots/tables**:

1. **cycles/IO vs qpairs** (shows multi-queue overhead scaling)
2. **DRAM bytes/IO vs qpairs** (payload vs metadata estimate)
3. **doorbells/IO vs qpairs and QD**
4. **poll/scans per completion vs qpairs** (or completions per poll-call)
5. (optional) **joules/IO vs IOPS** (host-side)

**Outcome.** Phase 1 provides a rigorous system-level justification for the I/O uncore architecture, independent of RTL or silicon assumptions.

## 1.8 Additional Phase 1 experiments and controls (recommended)

These experiments are not strictly required for the core claim, but they significantly strengthen attribution and reviewer confidence.

### 1.8.1 Multi-device scaling (2–4 SSDs if available)

**Goal:** show that host overhead scales with aggregate IOPS and queue count, not just with a single device.

- Run the same workload on 1 SSD vs 2 vs 4 (if hardware permits).
- Keep per-device IO size/QD constant; vary number of qpairs proportionally.
- Report: cycles/IO, DRAM bytes/IO, doorbells/IO, and total poller cores required.

### 1.8.2 NUMA / locality sensitivity (desktop/server only)

**Goal:** demonstrate fragility of software polling and the need for hardware execution.

- Run with poller cores and hugepages pinned on the same NUMA node as the SSD vs cross-node.
- Report p99 latency shifts and cycles/IO sensitivity.

### 1.8.3 Interrupt vs polling baseline (sanity and framing)

**Goal:** justify why the evaluation focuses on polling (interrupts collapse at high IOPS).

- Run a lower-IOPS configuration with interrupts enabled (if feasible).
- Compare CPU usage and p99 latency vs polling mode.
- Use this as a narrative anchor: “polling is the best-case software baseline.”

### 1.8.4 Buffer model sensitivity (pinned/hugepage vs fragmented)

**Goal:** validate the workload regime assumption and show robustness.

- Compare reused pinned hugepage buffers vs a less-ideal buffer allocation pattern.
- Report the change in DRAM metadata bytes/IO and cycles/IO.

### 1.8.5 Negative control: reduce qpairs while holding IOPS constant

**Goal:** separate “multi-queue scanning” from “raw IOPS.”

- Fix total IOPS target (by throttling) and vary qpairs.
- Expectation: cycles/IO and scans/completion should increase with qpairs.

## 1.9 Metric sufficiency check (what the Phase 1 metrics must answer)

By the end of Phase 1, the collected metrics must allow the paper to answer, quantitatively:

1. **Where do CPU cycles per I/O go?** (polling/scanning vs other work)
2. **How much DRAM traffic per I/O is metadata vs payload?**
3. **How do costs scale with qpairs, QD, and devices?**
4. **How many cores are required as aggregate IOPS increases?**

If any of these cannot be answered, add instrumentation until they can.

---

## 1.10 Optional cross-check: kernel NVMe / io_uring

Phase 1 is SPDK-first by design. However, one optional cross-check strengthens generality:

- Run an equivalent fio/io_uring workload (SQPOLL if available) at comparable IO size and QD.
- Compare qualitative trends (cycles/IO scaling with qpairs, DRAM metadata bytes/IO).

This is not required for the main claim, but preempts the critique "this is only SPDK."

# Phase 2: Virtual I/O Uncore Prototype (Required)

**Purpose.** Phase 2 validates—using real workloads—that the Phase-1 counterfactual holds when the software stack is actually running. Phase 2 is about **behavioral fidelity and measurable host-side deltas** (cycles/IO, DRAM bytes/IO, doorbells/IO, latency), **not** about ASIC power/area.

**Why Phase 2 is required.** Phase 1 motivates the architecture via measured overheads and counterfactual modeling, but Phase 2 is **necessary** to demonstrate that these benefits materialize under real software behavior. Without Phase 2, the evaluation would rely solely on modeling assumptions. Phase 2 provides concrete evidence that (a) the mechanism is implementable, (b) unmodified applications benefit, and (c) batching and readiness tradeoffs behave as predicted in practice.

---

## 2.0 Design goal for the prototype (keep it minimal)

The prototype should implement only what is required to validate the core thesis:

- **SRAM-resident semantics (architectural effect):** keep SQ/CQ head/tail and bookkeeping in private state; batch DRAM-visible updates.
- **Doorbell aggregation:** reduce device-facing doorbell frequency (count/time policies).
- **CQ suppression:** expose completions to software in batches (count/time policies).

Do **not** attempt to implement a full NVMe controller, full PCIe DMA engine, or IOMMU behavior. The proxy is allowed to be a **timing-abstracted execution engine**.

---

## 2.1 Two interaction modes to evaluate (must keep this explicit)

To be comprehensive and reviewer-proof, Phase 2 evaluates two modes:

**Mode 2A: Transparent mode (no SPDK changes)**

- SPDK applications and SPDK NVMe library are unchanged.
- SPDK continues to poll DRAM CQs as usual.
- The proxy/uncore changes *how DRAM-visible CQ/pointer state is updated* (batched exposure) and *how many doorbells reach the device*.

**What this demonstrates:** a **lower bound** on benefits from hardware interception and batched exposure alone.

**Mode 2B: Poll-lite mode (minimal SPDK NVMe library enablement)**

- SPDK applications remain unchanged.
- SPDK NVMe library is minimally patched to avoid scanning DRAM CQs when nothing is ready.
- SPDK polls an uncore "ready" signal (counter/bitmap) or uses interrupt-per-batch.

**What this demonstrates:** an **upper bound** on achievable host savings when software cooperates with the uncore's readiness interface.

---

## 2.2 Implementation approach (recommended plan)

There are two viable implementation paths; start with Option A for clarity and repeatability.

### Option A (recommended): QEMU virtual PCIe PF + uncore engine

**High-level idea.** Implement a QEMU PCIe device that exposes a small set of registers and queue resources. Internally, QEMU runs an "uncore engine" thread that:

- observes SQ writes / tail updates,
- schedules work,
- generates completions,
- and exposes them back to software in batches.

**Why QEMU is recommended.**

- Easy to inject batching policies and timing.
- Clean separation of "device/uncore behavior" vs "host software."
- Reproducible results (does not depend on SSD burstiness).

**Minimal device interface (keep it small).** Expose a BAR with:

- `UNC_CFG`: enable/disable, mode selection (2A vs 2B), batching knobs.
- `DB_IN[i]`: per-queue doorbell input (host writes tail updates here).
- `READY_BM`: readiness bitmap (poll-lite) — bit i means queue i has ≥1 completion ready.
- `READY_CNT[i]`: per-queue ready count (optional).
- (Optional) `CQ_MIRROR_BASE[i]`: host address for CQ mirroring.

**Queue state inside QEMU (acts like SRAM).** Maintain per-queue state:

- `sq_head`, `sq_tail_shadow`
- `cq_head`, `cq_tail_shadow`
- `pending_completions`
- `doorbell_pending`
- timers for `T` timeouts

**Backing model for "storage work."** Choose one, increasing fidelity:

1. **Synthetic completion model (recommended first):** configurable latency distribution, no payload.
2. **File-backed block device:** forward to a file + injected latency.
3. **Real SSD backing:** forward to a real device (noisy; later).

Start with (1) to validate mechanism; use (2) if reviewers demand "real IO."

### Option B: Kernel pseudo-device (fallback)

Implement a kernel module to emulate readiness/batching. Faster if kernel infrastructure exists, but harder to keep portable and clean. Prefer Option A unless kernel work is already planned.

---

## 2.3 Step-by-step build plan (student-friendly)

**Step 1 — Minimal queue engine (no NVMe yet).**

- Implement N queues in the proxy.
- Host writes `DB_IN[i] = new_tail`.
- Proxy updates `sq_tail_shadow`.
- Proxy schedules completions after fixed latency and appends to `pending_completions`.

**Step 2 — CQ suppression (batch exposure).**

- Add knobs: `CQ_BATCH_N` and `CQ_BATCH_T`.
- Publish CQEs to host-visible memory when `pending >= N` or timer ≥ `T`.

**Step 3 — Doorbell aggregation.**

- Add knobs: `DB_BATCH_B` and `DB_BATCH_T`.
- Absorb host doorbells; forward/commit work only every B or on timer.

**Step 4 — Transparent mode path (no SPDK changes).**

- Provide a DRAM-mirrored CQ ring visible to software.
- Ensure a polling loop can observe CQEs by reading that mirror.

**Step 5 — Poll-lite interface.**

- Maintain `READY_BM` / `READY_CNT[i]` based on pending completions.
- Clear readiness when batches are drained.

**Step 6 — Minimal SPDK NVMe library enablement (Mode 2B).**

- Patch only the polling loop to:
  - check `READY_BM` first,
  - only touch DRAM CQ for queues whose bit is set.

Keep the patch isolated (one function). Applications remain unchanged.

---

## 2.4 Running workloads (what counts as “unmodified apps”)

Phase 2 should run at least:

- `spdk_nvme_perf` (or `bdevperf`) for controlled sweeps
- an embedding-like microbenchmark (random reads of 4K/16K into reused buffers)

If using QEMU, workloads may run inside a VM (guest runs SPDK) or via a host harness, depending on exposure. The key requirement is that **application behavior is unchanged** (IO sizes, QD, qpair count).

---

## 2.5 Metrics to collect in Phase 2 (must align with Phase 1)

Collect the same must-have metrics as Phase 1 for direct comparison:

1. **IOPS and latency:** IOPS, p50/p99/p99.9.
2. **CPU cost:** cycles/IO, instructions/IO, CPU utilization.
3. **Polling/scanning intensity:** completion-check calls/sec, completions per call, scans per completion.
4. **DRAM traffic:** read/write bytes/IO; metadata estimate.
5. **Doorbells:** doorbells/IO.

Add Phase-2-specific diagnostics:

- average CQ publish batch size (N effective)
- average doorbell aggregation size (B effective)
- readiness bitmap hit rate (Mode 2B)

---

## 2.6 Experiment matrix for Phase 2

Keep the matrix small and interpretable:

1. Fix workload (4K random reads), fixed QD and qpairs.
2. Sweep `CQ_BATCH_N` ∈ {1, 4, 16, 64} at fixed `CQ_BATCH_T`.
3. Sweep `CQ_BATCH_T` ∈ {0, 2µs, 10µs, 50µs} at fixed `CQ_BATCH_N`.
4. Sweep `DB_BATCH_B` similarly.
5. Compare Mode 2A vs Mode 2B at the same knobs.

Outputs: latency–throughput trade curves and cycles/IO vs batching parameters.

---

## 2.7 Correctness criteria (must always hold)

- Every completion corresponds to exactly one submission.
- Command ordering within a queue preserved as required.
- No dropped CQEs; head/tail wrap handled correctly.
- Progress under low load via timeout.

---

## 2.8 Phase 2 success criteria (concrete)

Phase 2 is successful if:

- Mode 2A shows measurable reductions in cycles/IO and DRAM metadata bytes/IO at similar latency.
- Mode 2B shows substantially larger reductions, especially with many qpairs.
- Trade curves show clear p99 latency vs CPU cost behavior as N/T/B vary.

**Outcome.** Phase 2 validates the Phase-1 counterfactual with real software behavior and provides a rigorous lower/upper bound envelope for the uncore's impact before RTL work begins.

---

## 2.9 Additional Phase 2 experiments (recommended)

These experiments strengthen the argument that the prototype captures real-world behavior and that improvements are robust.

### 2.9.1 Multi-queue stress and scaling law validation

- Sweep qpairs more aggressively (e.g., 1 → 4 → 16 → 64 → 128) at fixed IO size and QD.
- Plot cycles/IO and scans/completion for Mode 2A vs Mode 2B.
- Goal: show that poll-lite eliminates scanning-overhead growth with qpairs.

### 2.9.2 Multi-device composition (QEMU)

- Instantiate 2–4 virtual devices and distribute qpairs.
- Goal: demonstrate that the uncore model composes cleanly as devices scale.

### 2.9.3 Backing-model sensitivity (synthetic vs file-backed vs optional real device)

- Run the same batching knobs under:
  - synthetic completion generator,
  - file-backed device,
  - optional real NVMe namespace.
- Goal: show that CPU/DRAM savings are not artifacts of the backing model.

### 2.9.4 Readiness interface quality (Mode 2B)

- Report readiness bitmap hit rate and false-work rate:
  - % of readiness checks that lead to draining ≥1 completion
  - average completions drained per ready queue
- Goal: prove poll-lite does not add significant overhead or misprediction.

### 2.9.5 Low-load and burst behavior (timeout correctness)

- Test low IOPS regimes and bursty regimes.
- Verify timeout-driven progress (T) bounds latency without harming throughput.

---

## Phase 3: RTL and Silicon Feasibility

**Purpose.** Phase 3 establishes that the I/O uncore hot path is **cheap, fast, and buildable in silicon**. This phase validates feasibility and cost; it does *not* re-justify the architectural motivation.

### 3.1 RTL scope (minimal by design)

RTL focuses only on hot-path control logic:

- SQ queue engine (head/tail updates, credit tracking, SQE decode pipeline)
- CQ engine (completion enqueue, moderation, batched writeback interface)
- Doorbell coalescer (aggregation FSM, counters, timers)
- Abstracted DMA sequencer (token-based issue + backpressure, no full PCIe)
- Optional lightweight scheduler (e.g., weighted round-robin)

Full NVMe, PCIe, and IOMMU implementations are explicitly out of scope.

### 3.2 SRAM banking analysis

Because the design is SRAM-dominated, feasibility hinges on SRAM bandwidth:

- Derive SRAM accesses per I/O (pointer updates, FIFO push/pop).
- Multiply by target IOPS (e.g., 30–40M IOPS/tile).
- Choose bank count and ports to meet bandwidth with margin.

Report:

- total SRAM size (MB)
- banks and ports
- maximum sustainable IOPS from SRAM limits

### 3.3 Synthesis and power estimation

- Synthesize RTL using standard-cell libraries (e.g., Synopsys DC/Fusion Compiler).
- Use compiled SRAM macros for realistic area and power.
- Estimate activity using traces or analytic toggle models derived from Phase 1/2.

Report:

- area breakdown (SRAM vs logic)
- maximum frequency
- added latency (cycles → ns)
- dynamic power and energy per I/O (nJ/IO)

### 3.4 Phase 3 success criteria

- Sustains target IOPS without excessive SRAM porting.
- Added latency bounded (O(100 ns)).
- Energy per I/O in the tens-of-nJ range.
- Area and power scale linearly with QPs and SRAM size.

**Outcome.** Phase 3 shows that the I/O uncore hot path is SRAM- and FSM-dominated, with modest area and power, supporting integration into a CPU I/O die.